

CS240 Algorithm Design and Analysis

Spring 2026

Problem Set 1

Due: 23:59, March 26, 2026

1. Please write your solutions in English.
2. Submit your solutions to **Gradescope**. When submitting, match your solutions to the problems correctly.
3. If you want to submit a handwritten version, scan it clearly.
4. Up to 2 late days are allowed for homework submissions. Assignments submitted within the first 24 hours after the deadline will incur a 10% penalty; those submitted within 48 hours will incur a 20% penalty. Submissions more than 48 hours late will not be accepted unless there is a valid reason, such as a medical or family emergency.
5. You are required to follow ShanghaiTech's academic honesty policies. You are allowed to discuss problems with other students, but you must write up your solutions by yourselves. You are not allowed to copy materials from other students or from online or published resources. Violating academic honesty can result in serious penalties.

Problem 1:

Sort the following functions in ascending order of growth.

$$f_1(n) = 2025^{2026^n} \quad (1)$$

$$f_2(n) = 2026^{2025^n} \quad (2)$$

$$f_3(n) = \log_2 8^n \quad (3)$$

$$f_4(n) = n^{\log_2 8} \quad (4)$$

$$f_5(n) = 2027^{2026^{2025}} \quad (5)$$

$$f_6(n) = 2^{1.1 \log_2 n} \quad (6)$$

$$f_7(n) = n^{\log_2 n} \quad (7)$$

$$f_8(n) = 2^{\log_2 \log_2 n} n \quad (8)$$

$$f_9(n) = C_n^{n/2} \quad (9)$$

Hint: $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$.

Solution:

Problem 2:

Determine the time complexity of the pseudocode below and explain your reasoning.

```
1  def count_triples(arr, threshold):
2      n = len(arr)
3      count1 = 0
4      count2 = 0
5
6      def merge_sort(arr):
7          if len(arr) <= 1:
8              return arr
9
10         mid = len(arr) // 2
11         left = merge_sort(arr[:mid])
12         right = merge_sort(arr[mid:])
13
14         result = []
15         i = j = 0
16
17         while i < len(left) and j < len(right):
18             if left[i] <= right[j]:
19                 result.append(left[i])
20                 i += 1
21             else:
22                 result.append(right[j])
23                 j += 1
24
25         result.extend(left[i:])
26         result.extend(right[j:])
27
28         return result
29
30     arr = merge_sort(arr)
31
32     for i in range(n):
33         for j in range(i + 1, n):
34             k = j + 1
35             while k < n and arr[i] + arr[j] + arr[k] <= threshold:
36                 count1 += 1
37                 k += 1
```

```
38
39     for i in range(n):
40         left = i + 1
41         right = n - 1
42
43         while left < right:
44             if arr[i] + arr[left] + arr[right] <= threshold:
45                 count2 += (right - left)
46                 left += 1
47             else:
48                 right -= 1
49
50     return count1 + count2
```

Solution:

Problem 3:

Given a set of n points on the real axis $A = \{A_1, \dots, A_n\}$ ($A_i \in \mathbb{R}$). Please design a greedy algorithm to find **the minimum number of unit-intervals to cover all the given points**. That is to say, each point is covered by at least one interval.

- A **unit-interval** is a closed interval $[l, r]$ where $r - l = 1$.
- A point x is **covered** by an interval $[l, r]$ if and only if $l \leq x \leq r$.

Example: Given $A = \{1.01, 10.1, 3.14, 2.33\}$. Then the minimum number of unit-intervals needed is 3 and $[1.0, 2.0] \cup [2.3, 3.3] \cup [9.5, 10.5]$ are 3 possible unit-intervals covering all points in A .

Note: Your solution should include the following three parts:

- Describe your algorithm in **pseudocode** or **natural language**.
- Proof of correctness of your algorithm using **exchange argument**.
- Time complexity analysis. Your algorithm should run in $o(n^2)$ time.

Solution:

Problem 4:

You are driving on a highway represented by an array of fuel stations. You start at the first station. Each station in the array has a number representing how far you can drive from that station before needing to stop at another one for fuel. Assume you always have enough fuel to reach the last station. Your goal is to reach the last station using the smallest possible amount of refuel. Please provide a greedy solution for this scenario.

Example:

- **Input:** `stations = [2, 3, 0, 1, 4]`
- **Output:** 2
- **Explanation:** The minimum number of refuels to reach the last station is 2. Refuel at station 0 to drive to station 1, then refuel at station 1 to drive directly to the last station.

Solution:

Problem 5:

You are given an $n \times n$ grid, and a procedure $V(i, j)$ that assigns an integer value to each position (i, j) in the grid, where i and j are integers such that $1 \leq i, j \leq n$. Your goal is to find a local minimum (or sink) in the grid (i.e., integers i^*, j^* with $1 \leq i^*, j^* \leq n$ such that for all neighbors (i, j) of (i^*, j^*) in the grid, $V(i^*, j^*) \leq V(i, j)$). The neighbors of (i, j) are $(i - 1, j)$, $(i + 1, j)$, $(i, j + 1)$, $(i, j - 1)$; the elements along the diagonals do not count as neighbors.

Design an algorithm that makes $O(n)$ calls to V . Note that the grid has n^2 nodes.

Solution:

Problem 6:

In a city rich with history, there is an old town that was once the heart of the city. However, as time passed, the streets and buildings of the old town fell into disrepair, and the layout became chaotic. The streets in this old town are connected like a linked list, one after another, and each street has a unique number representing its construction period and architectural style. Over the years, these numbers have fallen out of order, leading to inconvenient traffic, dilapidated buildings, and a decline in the quality of life for the residents. To revitalize the old town, the city government has launched the “Old Town Street Reconstruction Plan”. The planners aim to restore the streets to their former glory by arranging them in an orderly sequence according to their numbers, making the layout more logical and facilitating better traffic flow and daily life. Your task is to act as the engineer for this reconstruction project, responsible for reordering these streets.

Given a linked list representing the structure of the streets, where the head of the list is *head*, you are required to use an efficient divide-and-conquer algorithm to sort all the street numbers in ascending order. The reconstructed streets must be sequentially connected, restoring the old town’s orderly beauty. Please return the sorted linked list of streets.

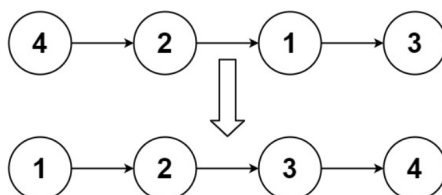


Figure 1: For example

For example, the street numbers of the old town are currently arranged as $[4, 2, 1, 3]$, with the head node $head = 4$. You need to sort them in ascending order as $[1, 2, 3, 4]$.

Solution: